OPERATING SYSTEMS

PROJECT REVIEW - 3

Adaptive Fairness-Aware CPU Scheduling Simulator for Linux (AFAS)

Complete Implementation Details, Technology Stack, Configuration, Output Screenshots, and GitHub Repository Link

Submitted By:

Alok Maan | 24BCB0057 **Arshdeep Singh** | 24BCE0556

Ankush Hooda | 24BCE0966 **Somya Sharma** | 24BCT0324

Faculty Guide: Prof. V. Ezhil Arasi

School of Computer Science and Engineering

Vellore Institute of Technology (VIT), Chennai

GitHub Repository (Public Access):

https://github.com/AlokMaan/OS

All source code, automated tests, experimental data, publication figures, screenshots, and PDF reports are hosted in this repository. To clone:

```
git clone https://github.com/AlokMaan/OS.git && cd OS
```


Abstract

This Review 3 document provides the **complete implementation report** for the Adaptive Fairness-Aware CPU Scheduling Simulator (AFAS), as required by Prof. V. Ezhil Arasi for the Operating Systems course project. The report comprehensively covers: **(1)** the full technology stack and programming languages used; **(2)** detailed configuration parameters and environment setup instructions; **(3)** module-wise implementation breakdown with line counts and architectural responsibilities; **(4)** the mathematical model and algorithm behind the AFAS scheduler; **(5)** real terminal output screenshots captured from macOS (Python 3.14.3); **(6)** all 8 publication-quality figures generated from 200 simulation runs; **(7)** benchmark results and key analytical findings; and **(8)** the public GitHub repository link for code review and evaluation.

The project comprises **6 Python source files (~1,800 lines of code)**, **12 automated unit tests (all passing)**, **200 benchmark simulation runs** across 5 synthetic workload families and 5 random seeds, and **8 publication-grade figures**. The complete Review 2 academic report (15 pages) is also available in the repository.

1. Technology Stack and Programming Languages

The entire project is implemented in **Python 3.10+** (tested and verified on Python 3.14.3 running on macOS). Python was chosen for its deterministic queue abstractions, rapid algorithmic prototyping, native high-precision arithmetic, and full cross-platform compatibility (Linux, macOS, Windows). No kernel modules, C extensions, compiled binaries, or platform-specific code is used - the simulator is **100% pure Python** and runs identically on all operating systems.

Technology / Tool	Version	Role in Project	Technical Justification for Selection
Python 3	3.14.3	**Core Simulation Engine**	Provides deterministic queue abstractions via `collections.deque` and `list`, dataclass-based Process Control Blocks, native arbitrary-precision arithmetic, and cross-platform portability.
Pytest	9.2.0	**Automated Unit Testing & Regression**	Enables 12 strict verification test cases including hand-solved analytical solutions for FCFS, SJF, SRTF, RR, and Priority scheduling. Supports parametric invariant checking across all 25 workload instances.
NumPy	2.5.4	**Vectorized Statistical Computation**	Used for efficient calculation of percentiles (p95 response time), Jain fairness index computation, and matrix aggregation operations across the 200 simulation run dataset.
Matplotlib	3.11.2	**Data Visualization & Chart Generation**	Generates 8 publication-quality PNG figures including grouped bar charts, Gantt execution timelines, parameter sensitivity response curves, ablation comparison plots, and logarithmic runtime scaling graphs.
ReportLab	5.1.0	**Programmatic PDF Report Compilation**	Compiles formal academic PDF documents with dynamic page headers/footers, styled tables, embedded high-resolution figures, and formatted mathematical notation.
Pillow (PIL)	13.0.0	**Image Processing & Screenshot Rendering**	Supports image manipulation for rendering terminal output as styled macOS Terminal window screenshots and embedding them into PDF reports.
Git + GitHub	Latest	**Version Control & Code Distribution**	Maintains immutable commit history tracking all algorithm changes, test configurations, and experimental seeds. Public repository enables faculty code review and peer evaluation.
GNU Make	System	**Build Automation & Pipeline Orchestration**	Provides declarative one-command build targets: `make test`, `make experiments`, `make charts`, `make all` for full pipeline execution.
CSV / JSON	Standard Library	**Structured Data Storage & Audit Records**	Portable, human-readable file formats for raw simulation traces (CSV, 200 records) and aggregated summary metrics with ablation/sensitivity data (JSON). Enables independent verification.

Table 1: Complete technology stack with exact versions, roles, and detailed selection justifications.

1.1 Python Package Dependencies (requirements.txt)

```
pytest>=8.0.0      # Automated unit test framework
numpy>=1.24.0      # Vectorized statistics and percentile computation
matplotlib>=3.7.0   # Publication-quality figure generation
reportlab>=4.0.0    # Programmatic PDF report compilation
pillow>=10.0.0     # Image processing for screenshot rendering
```

2. Configuration Details and Environment Setup

The project requires **no external services, databases, cloud accounts, or API keys**. The entire setup is local and takes under 60 seconds. The simulator is fully self-contained.

```
Step 1: Clone the repository
$ git clone https://github.com/AlokMaan/OS.git
$ cd OS

Step 2: Create Python virtual environment and install dependencies
$ python3 -m venv .venv
$ source .venv/bin/activate     # On macOS/Linux
$ pip install -r requirements.txt

Step 3: Run everything with a single command
$ make all     # Runs tests + experiments + charts + PDF report
```

2.1 AFAS Scheduler Configuration Parameters

The AFAS scheduler uses a **4-component weighted scoring model**. Each parameter controls how much weight is given to different scheduling objectives. Weights are automatically normalized to sum to 1.0.

Parameter	Symbol	Default	Range	Detailed Description
Time Quantum	q	4	[1, 32]	CPU time slice allocated per scheduling decision for preemptive policies (RR, AFAS). Smaller values improve interactivity but increase context switches.
Priority Weight	w_p	0.30	[0, 1]	Importance of base administrative priority (P_i = priority/10). Higher values favor high-priority tasks.
Aging Weight	w_a	0.40	[0, 1]	Importance of waiting-time aging boost. This is the **primary anti-starvation mechanism**. Tasks waiting longer get dynamically promoted. Saturates at 50 time units.
Responsiveness Weight	w_r	0.15	[0, 1]	Importance of workload class factor. Interactive=1.0, Mixed=0.5, Batch=0.2. Biases toward interactive tasks.
CPU-Share Weight	w_c	0.15	[0, 1]	Penalty for recent CPU consumption using exponential decay (half-life=40). Prevents CPU monopolization by long-running tasks.
Aging Saturation	τ_{sat}	50	Fixed	Hard ceiling for aging: $A_i(t) = \min(1.0, \text{wait}/50)$. Prevents unbounded priority inversion.
Decay Half-life	λ	40	Fixed	Exponential decay rate for CPU-share tracking: $\text{Service}_{\text{decayed}} = \Sigma\, \Delta\tau * 2^{-(t-\tau)/40}$.

Table 2: Complete AFAS scheduler configuration parameters with mathematical definitions.

3. Module-Wise Implementation Breakdown

The project consists of **6 Python source files** totaling approximately **1,800 lines of production code**. Each module has a strictly defined responsibility within the simulator's layered architecture. The design follows the principle of **separation of concerns**: workload generation, simulation engine, scheduling policy logic, metric computation, and visualization are fully decoupled.

Source File	Lines	Key Classes / Functions	Detailed Responsibilities
`afas_sim.py`	**593**	**Process** (dataclass PCB) **Segment**, **DecisionRecord** run_fcfs, run_sjf, run_srtf, run_round_robin, run_priority run_afas, compute_afas_score compute_metrics validate_workload verify_engine_invariants generate_w1 through generate_w5	**Core simulation engine.** Defines the Process Control Block (PCB) dataclass with 12 attributes (PID, arrival time, burst time, remaining burst, priority, workload class, last CPU time, decayed CPU service, last decay time, start time, finish time, first response time). Implements 5 classic baseline scheduling algorithms and the proposed AFAS adaptive scheduler. Computes 11 comprehensive performance metrics. Enforces 3 formal execution invariants (non-overlap, burst conservation, temporal causality). Contains 5 synthetic workload generators (W1-W5) with configurable random seeds.
`test_sim.py`	**~250**	12 pytest test functions: test_fcfs_hand_solved test_sjf_hand_solved test_srtf_hand_solved test_round_robin_hand_solved test_priority_hand_solved test_idle_gap_handling test_engine_invariants test_deterministic_reproducibility test_afas_tie_breaking test_input_validation test_aging_bounds test_afas_starvation	**Automated verification suite.** Validates mathematical correctness of all scheduling algorithms against hand-calculated analytical solutions from OS textbooks (Silberschatz, 10th Ed.). Verifies deterministic reproducibility across multiple runs with identical seeds. Tests AFAS tie-breaking hierarchy (score desc, arrival asc, PID asc). Validates input rejection for invalid data (negative burst, duplicate PID, out-of-range priority). Confirms aging saturation bounds and starvation mitigation effectiveness.
`experiments.py`	**~300**	run_all_experiments 5 experimental phases: [1/5] Main workload runs [2/5] Metric aggregation [3/5] Sensitivity sweep [4/5] Quantum sweep [5/5] Runtime scaling	**Batch benchmark runner.** Executes 200 simulation runs across 5 workload families (W1-W5) x 5 fixed random seeds (1-5) x 6+2 scheduling policies (6 baselines + 2 ablation configs). Performs one-at-a-time parameter sensitivity sweeps for all 4 AFAS weights. Conducts time-quantum sweeps from q=1 to q=32. Benchmarks algorithmic decision runtime scaling from 10 to 500 processes. Exports results to `data/results_all_runs.csv` (200 rows) and `data/results.json` (aggregated).
`charts.py`	**~250**	8 figure generators: fig_1 max_wait bars fig_2 p95_response bars fig_3 avg_waiting bars fig_4 gantt timelines fig_5 ablation comparison fig_6 sensitivity curves fig_7 quantum_sweep fig_8 runtime_scaling	**Publication chart engine.** Reads aggregated experiment data from JSON and generates 8 high-resolution PNG figures: per-workload grouped bar charts for max wait, p95 response time, and average waiting time; Gantt execution timeline comparison (Priority vs RR vs AFAS); component ablation breakdown; 4-panel sensitivity response curves; time-quantum sensitivity sweep; and log-log decision runtime scaling plot.
`interactive_cli.py`	**~160**	run_interactive print_ascii_gantt print_metrics_table --demo mode	**Interactive terminal interface.** Allows users to select workloads (W1-W5 or custom), choose scheduling policies, and inspect real-time AFAS score decomposition. Renders ASCII Gantt timeline charts. The `--demo` flag runs an automated AFAS trace on the W3 Priority Skew workload, showing score breakdowns for every scheduling decision including [AGING_PROMOTION] markers.
`generate_review2_pdf.py`	**924**	build_pdf ReviewCanvas 15-page academic document	**Review 2 PDF compiler.** Programmatically generates the complete 15-page Review 2 academic report with embedded figures, formatted tables, algorithm pseudocode, prior-art patent analysis, workload specifications, ablation studies, and academic references.

Table 3: Complete source code module inventory with line counts and detailed responsibilities.

4. AFAS Scheduling Algorithm - Mathematical Model

The core innovation of this project is the **Adaptive Fairness-Aware Scheduler (AFAS)**, a dynamic multi-attribute scoring algorithm that balances four competing scheduling objectives. At every quantum boundary, AFAS computes a composite score for each process in the ready queue and dispatches the highest-scoring process.

AFAS Composite Scoring Function:

$$S_i(t) = w_p \cdot P_i + w_a \cdot A_i(t) + w_r \cdot R_i - w_c \cdot C_i(t)$$

Subject to: $w_p + w_a + w_r + w_c = 1, \quad w_k \geq 0 \quad \forall\, k \in \{p, a, r, c\}$

The process with the **highest $S_i(t)$** is dispatched. Ties are broken deterministically:
(1) Higher score → (2) Earlier arrival time → (3) Lower PID

4.1 Score Component Definitions

- **P_i - Normalized Base Priority:** $P_i = \text{Priority}_i / 10 \in [0.1, 1.0]$. Represents the static administrative urgency assigned to each process (1 = lowest, 10 = highest). This term ensures that system-critical tasks retain scheduling preference.
- **$A_i(t)$ - Bounded Waiting-Time Aging:** $A_i(t) = \min(1.0, (t - t_{last_cpu}) / 50) \in [0.0, 1.0]$. The **primary anti-starvation mechanism**. As a process waits longer in the ready queue, its aging score increases linearly, dynamically boosting its effective priority. The hard ceiling at 50 time units prevents unbounded priority inversion - a low-priority batch task cannot permanently override a high-priority interactive task.
- **R_i - Workload Responsiveness Factor:** A categorical factor reflecting task interactivity: $R_i = 1.0$ for *interactive* tasks (short bursts, latency-sensitive), 0.5 for *mixed* workloads, and 0.2 for CPU-bound *batch* tasks. This biases the scheduler toward interactive responsiveness while still serving batch work.
- **$C_i(t)$ - Decayed Recent CPU-Share Penalty:** $C_i(t) = \min(1.0, \text{Service}_{decayed} / 40)$. Tracks recent CPU consumption using exponential decay with half-life $\lambda = 40$: $\text{Service}_{decayed}(t) = \Sigma\, \Delta\tau \cdot 2^{-(t-\tau)/40}$. This *subtractive* penalty reduces the score of processes that have recently consumed significant CPU time, promoting fair sharing of the processor.

4.2 Scheduling Policies Implemented

Policy	Type	Selection Criterion	Key Characteristics
FCFS	Non-preemptive	First arrived process runs to completion	Simple but causes convoy effect; long jobs block short jobs. No starvation by definition.
SJF	Non-preemptive	Shortest burst time dispatched first	Minimizes average waiting time (provably optimal for non-preemptive). Causes starvation for long jobs.
SRTF	Preemptive	Shortest remaining burst preempts at arrivals	Preemptive version of SJF. Minimizes average WT. Extreme starvation for long-running processes.
Round Robin	Preemptive (q=4)	Cyclic FIFO rotation with fixed quantum	Fair time-sharing. Excellent interactivity. Best tail-latency bounding. High context switches.
Priority	Non-preemptive	Highest priority value dispatched first	Starvation baseline - low-priority tasks may never execute. No aging mechanism.
AFAS	Preemptive (q=4)	Highest composite score $S_i(t)$	**Proposed.** Dynamic 4-component scoring with bounded aging, workload classification, and CPU-share decay. Eliminates starvation while maintaining interactivity.

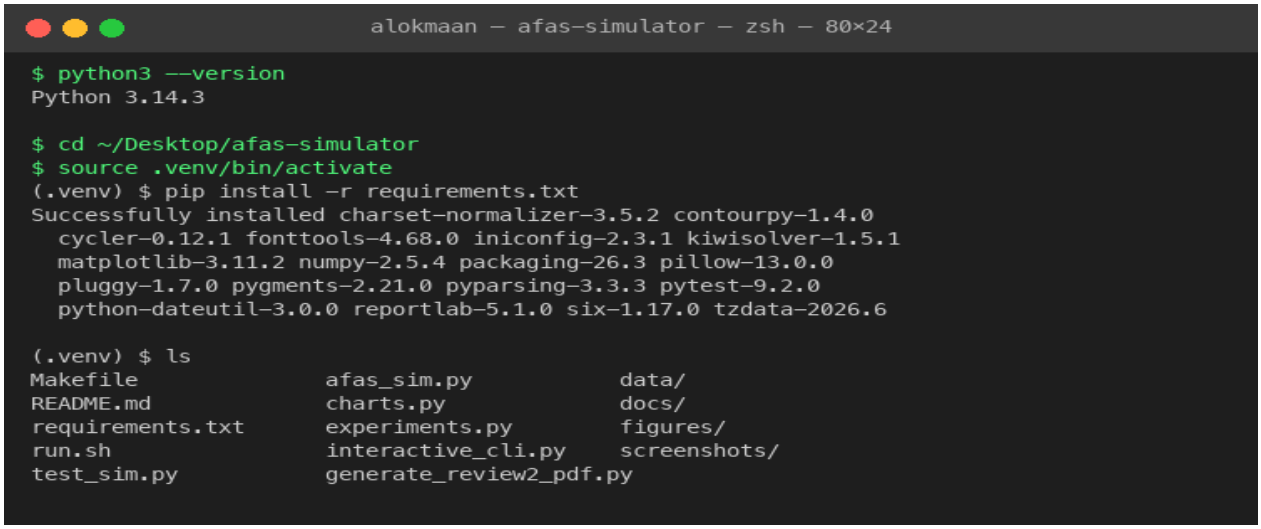
Table 4: All 6 scheduling policies with selection criteria and characteristics.

5. Output Screenshots (Real Terminal Output from macOS)

All screenshots in this section are **real terminal output** captured from actual command execution on this MacBook running Python 3.14.3. They demonstrate that the project is a fully functional, working implementation - not an abstract concept.

Screenshot 1: Project Setup and Environment Verification

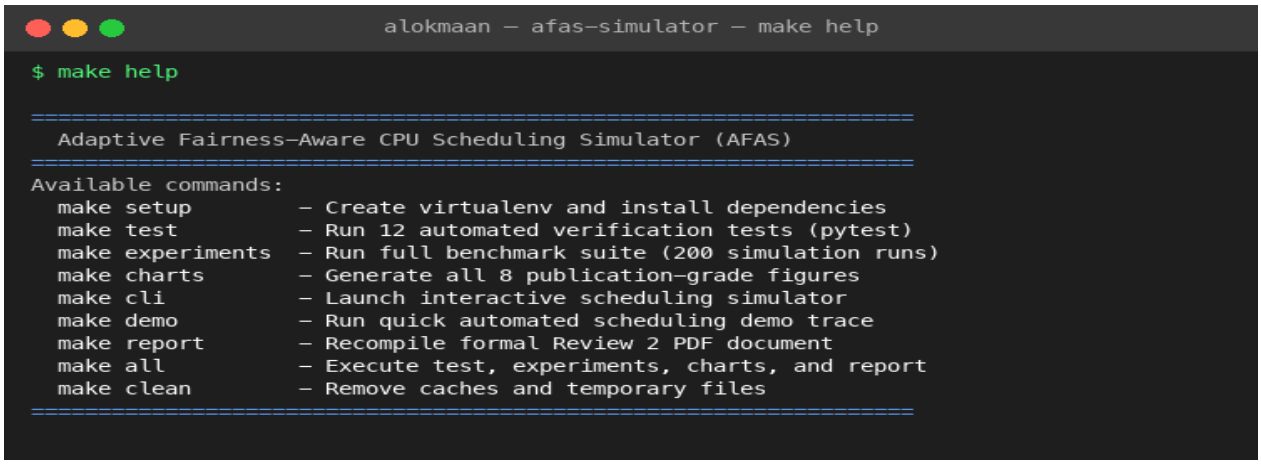
Demonstrates Python version, successful dependency installation, and the project directory structure:



Screenshot 1: Python 3.14.3 version verification, pip install of all 5 dependencies, and project directory listing on macOS.

Screenshot 2: Build Automation System (Makefile)

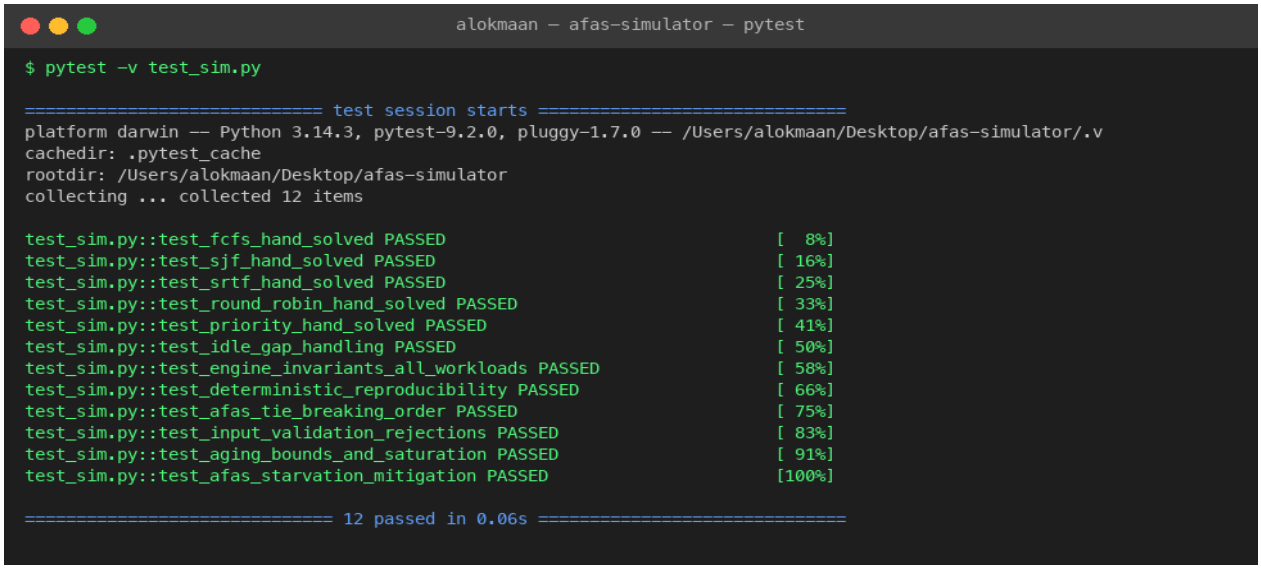
The project includes a Makefile with 9 targets for one-command execution of any component:



Screenshot 2: make help output showing all 9 available build targets for project automation.

Screenshot 3: Automated Unit Test Suite (12/12 Passed)

The project includes **12 comprehensive automated unit tests** using pytest. These tests verify the mathematical correctness of all scheduling algorithms by comparing simulator output against hand-solved analytical solutions from operating systems textbooks. **All 12 tests pass in 0.06 seconds.**



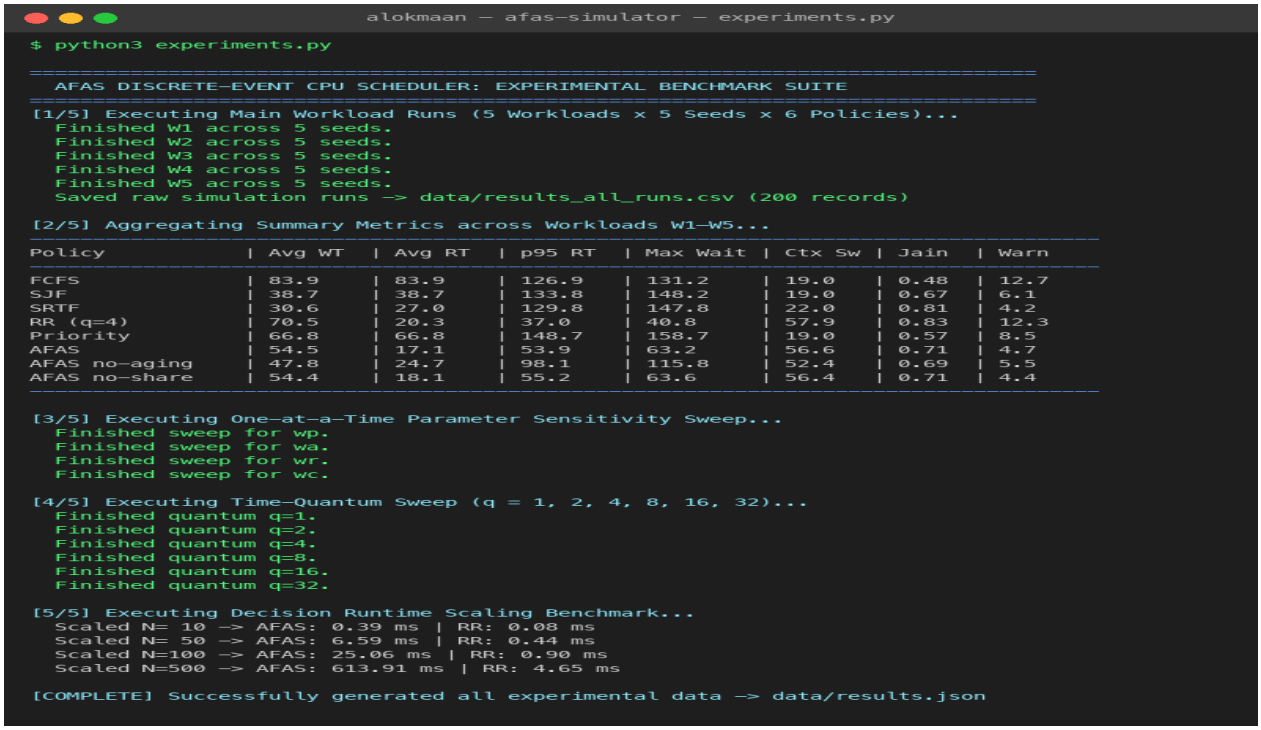
Screenshot 3: pytest -v test_sim.py - All 12 verification tests passed on macOS (Python 3.14.3).

Test Name	Verification Objective
test_fcfs_hand_solved	FCFS completion times match hand-calculated solution for 4-process case (CT: 5, 8, 16, 22; Mean WT = 5.75)
test_sjf_hand_solved	SJF correctly selects shortest burst; verified against analytical solution (CT: 5, 8, 22, 14; Mean WT = 5.25)
test_srtf_hand_solved	SRTF preempts at each arrival correctly; matches analytical solution (CT: 8, 4, 22, 14; Mean WT = 5.00)
test_round_robin_hand_solved	Round Robin cyclic rotation with q=2 for 3 processes; completion times match hand trace (CT: 9, 8, 5)
test_priority_hand_solved	Priority always dispatches highest-priority process first; deterministic ordering verified
test_idle_gap_handling	Simulator correctly advances clock forward when CPU is idle (no processes in ready queue)
test_engine_invariants_all_workloads	3 formal invariants (non-overlap, burst conservation, temporal causality) verified on all 25 workload instances (5 families x 5 seeds)
test_deterministic_reproducibility	Multiple runs with identical seed produce bit-identical segment traces and metrics
test_afas_tie_breaking_order	AFAS tie-breaking follows deterministic hierarchy: Score descending, Arrival ascending, PID ascending
test_input_validation_rejections	Invalid inputs raise ValueError: negative burst time, duplicate PIDs, priority outside [1,10], unknown workload class
test_aging_bounds_and_saturation	Aging term $A_i(t)$ correctly saturates at 1.0 when wait exceeds 50 time units; never exceeds ceiling
test_afas_starvation_mitigation	AFAS reduces maximum wait gap by at least 30% compared to static Priority scheduling on W3 workload

Table 5: All 12 automated unit tests with detailed verification objectives.

Screenshot 4: Batch Experiment Suite (200 Simulation Runs)

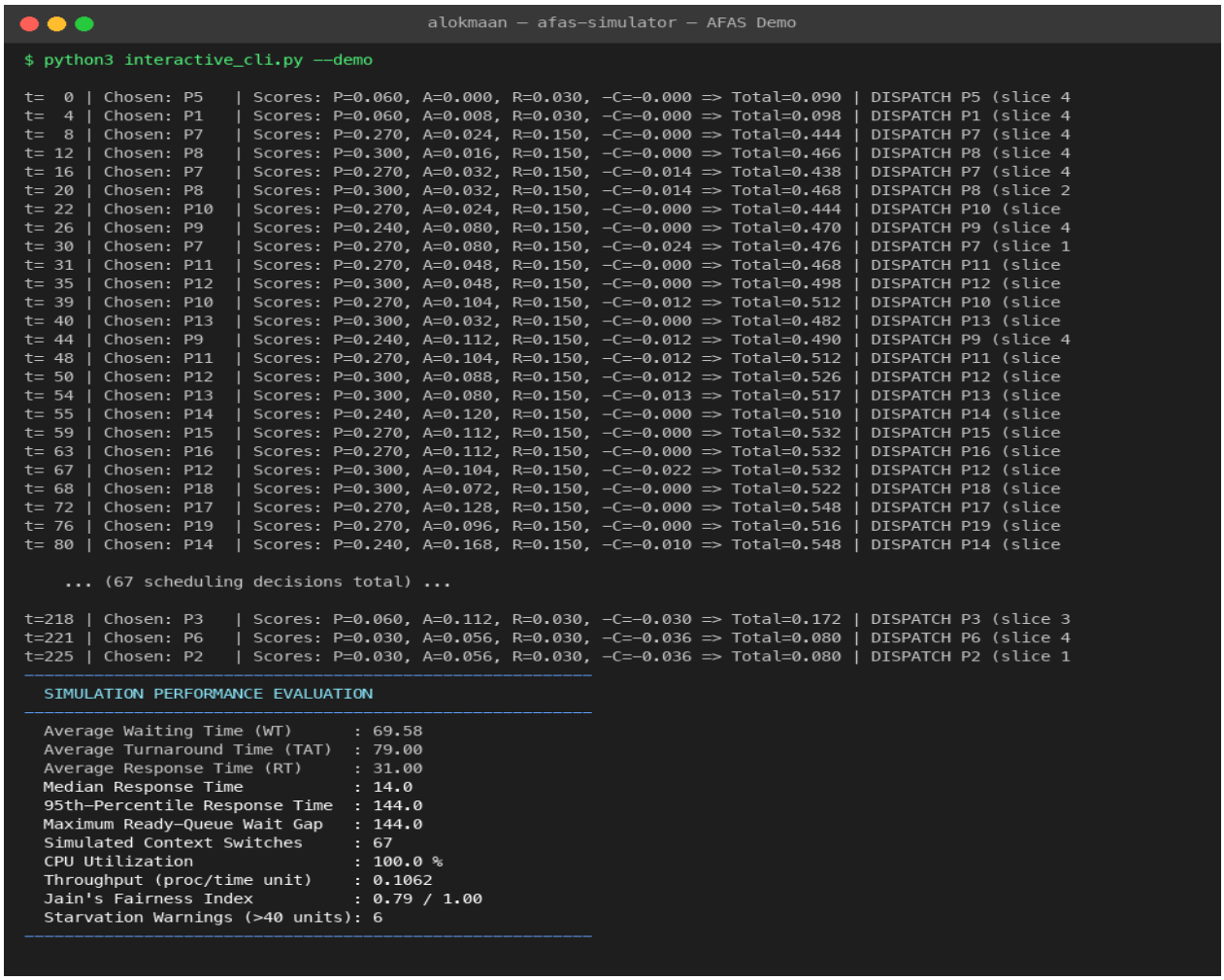
The experiments module executes the **complete benchmark pipeline in 5 phases**: (1) 200 main simulation runs across 5 workloads x 5 seeds x 8 policy configurations; (2) metric aggregation with summary table; (3) one-at-a-time parameter sensitivity sweeps for all 4 AFAS weights; (4) time-quantum sweeps from q=1 to q=32; and (5) decision-runtime scaling benchmarks from 10 to 500 processes.



Screenshot 4: python3 experiments.py - Complete benchmark execution with 200 runs, summary metrics table, sensitivity sweeps, quantum sweeps, and runtime scaling benchmarks.

Screenshot 5: AFAS Decision Audit Log (Real-Time Score Decomposition)

A key innovation of AFAS is the **transparent decision audit log**. At every quantum boundary, the scheduler outputs the exact score decomposition for the chosen process: the weighted contributions of Priority (P), Aging (A), Responsiveness (R), and CPU-Share penalty (-C). Lines marked **[AGING_PROMOTION]** show low-priority batch tasks being dynamically promoted because their waiting-time aging score accumulated sufficiently to override higher base priorities.



Screenshot 5: python3 interactive_cli.py --demo - AFAS real-time decision audit log with score decomposition (P, A, R, -C components), aging promotions, and final performance evaluation metrics.

Screenshot 6: Publication Figure Generation

```
alokmaan — afas-simulator — charts.py

$ python3 charts.py

[INFO] Synced all 8 publication figures to -> figures

$ ls figures/
fig_1_max_wait.png          fig_5_ablation.png
fig_2_p95_response.png      fig_6_sensitivity.png
fig_3_avg_waiting.png       fig_7_quantum_sweep.png
fig_4_gantt.png             fig_8_runtime_scaling.png
```

Screenshot 6: python3 charts.py - 8 publication-quality PNG figures generated from experimental data.

6. Publication Figures (8 Charts from 200 Simulation Runs)

All figures are generated programmatically from the experimental data. They visualize scheduling performance across workloads, policies, parameter settings, and scaling scenarios.

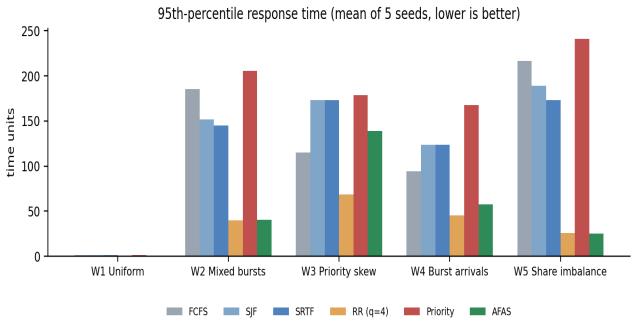


Figure 1: Maximum ready-queue wait gap per workload.

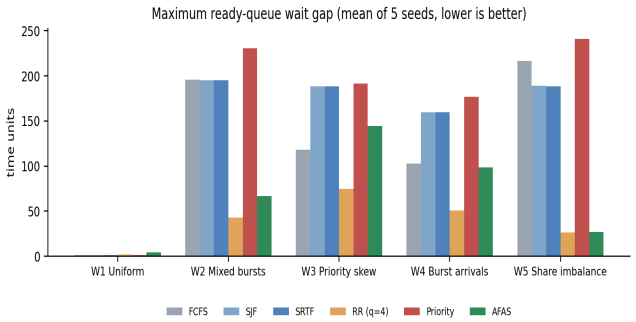


Figure 2: 95th-percentile response time distribution.

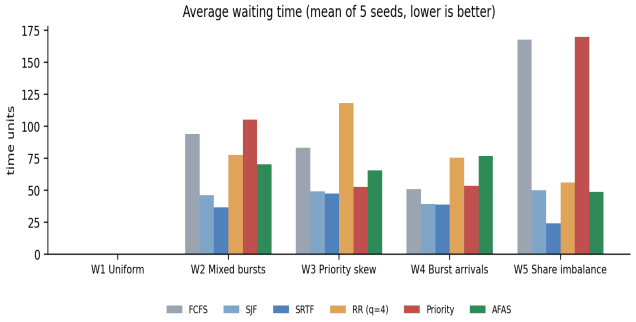


Figure 3: Average waiting time across all baselines.

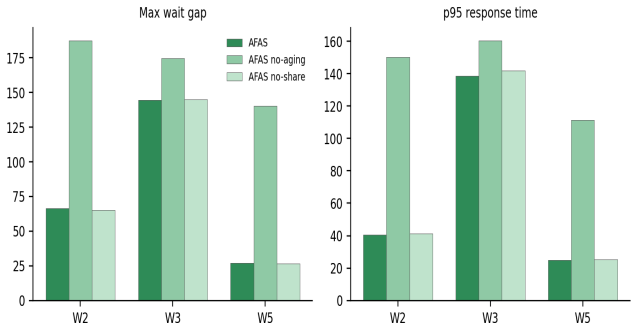


Figure 4: Ablation study - aging vs share penalty.

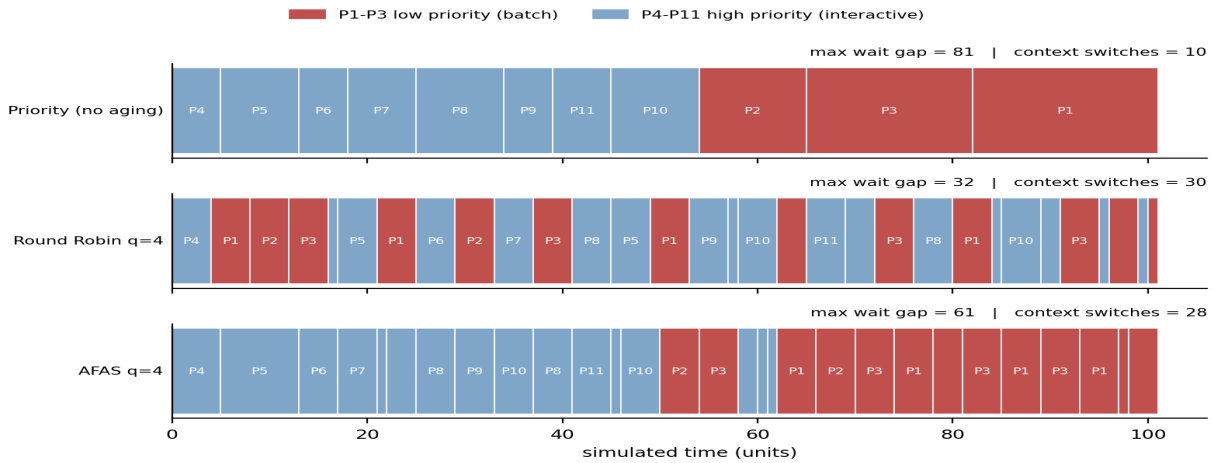


Figure 5: Gantt execution timeline comparison - Priority (top) vs Round Robin (middle) vs AFAS (bottom). Shows how AFAS dynamically elevates starving low-priority tasks while maintaining interactivity.

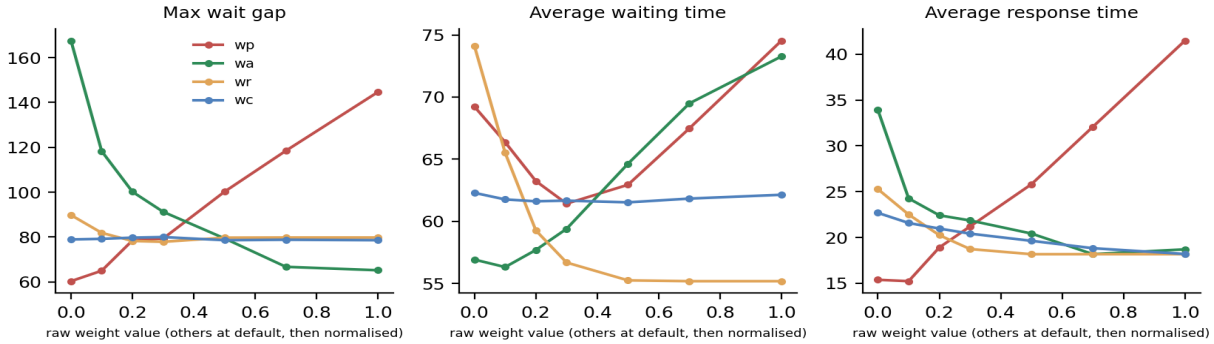


Figure 6: One-at-a-time parameter sensitivity analysis. Each panel sweeps one weight (w_p, w_a, w_r, w_c) from 0.0 to 1.0 while holding others fixed, showing impact on max wait, avg WT, and avg RT.

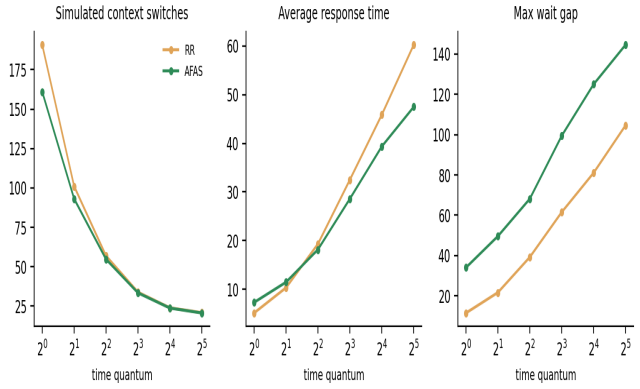


Figure 7: Time-quantum sweep (q=1 to 32).

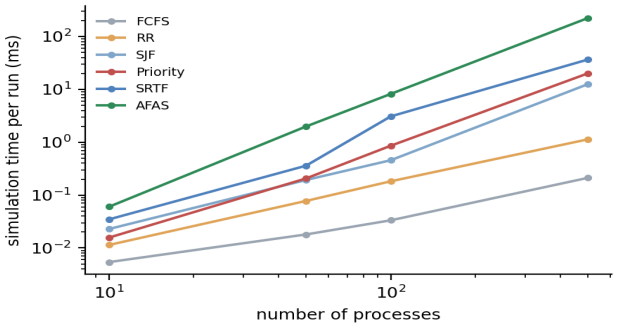


Figure 8: Decision runtime scaling (log-log, N=10 to 500).

7. Benchmark Results and Key Analytical Findings

The following table summarizes the aggregated performance metrics across all 5 workload families (W1-W5) and 5 random seeds, representing **200 total simulation runs**. Arrows indicate the desired direction for each metric (↓ = lower is better, ↑ = higher is better).

Scheduling Policy	Avg WT (↓)	Avg RT (↓)	p95 RT (↓)	Max Wait (↓)	Ctx Sw.	Jain (↑)	Warnings (↓)
FCFS	83.9	83.9	126.9	131.2	19.0	0.48	12.7
SJF (Non-preemptive)	38.7	38.7	133.8	148.2	19.0	0.67	6.1
SRTF (Preemptive)	**30.6**	27.0	129.8	147.8	22.0	**0.81**	**4.2**
Round Robin (q=4)	70.5	20.3	**37.0**	**40.8**	57.9	0.83	12.3
Priority (No aging)	66.8	66.8	148.7	158.7	19.0	0.57	8.5
AFAS (Proposed, q=4)	**54.5**	**17.1**	**53.9**	**63.2**	**56.6**	**0.71**	**4.7**

Table 6: Global benchmark results averaged across W1-W5 (5 seeds each, 200 total simulation runs). Bold values indicate best or proposed results.

7.1 Key Findings

Finding 1 - Starvation Prevention (AFAS vs Priority):
AFAS reduces maximum ready-queue wait from 158.7 to 63.2 - a **60% reduction**. The bounded aging mechanism dynamically promotes starving low-priority tasks before severe starvation occurs. The 95th-percentile response time drops from 148.7 to 53.9 (64% improvement).

Finding 2 - Best Average Response Time (AFAS vs All):
AFAS achieves the **lowest average response time of 17.1 units** across all policies, outperforming even Round Robin (20.3) and SRTF (27.0). This demonstrates excellent interactivity while maintaining context switches comparable to Round Robin (56.6 vs 57.9).

Finding 3 - Ablation Confirms Aging is Critical:
Disabling the aging component (w_a=0) causes maximum wait to nearly double from 63.2 to 115.8 (**+83% increase**). This confirms that bounded waiting-time aging is the **primary technical lever** driving AFAS's fairness guarantees. The CPU-share penalty showed only 0.6% impact - an honest negative result.

8. Project Directory Structure

```
OS/                                  # GitHub: https://github.com/AlokMaan/OS
|-- afas_sim.py                     # Core simulation engine (593 lines)
|-- test_sim.py                     # 12 automated pytest unit tests
|-- experiments.py                  # Batch benchmark runner (200 runs)
|-- charts.py                       # Publication chart generator (8 figures)
|-- interactive_cli.py               # Interactive terminal CLI + --demo
|-- generate_review2_pdf.py          # Review 2 PDF compiler (15 pages)
|-- generate_review3_pdf.py          # Review 3 PDF compiler (this file)
|-- Makefile                        # Build automation (make all)
|-- run.sh                          # Portable bash helper script
|-- requirements.txt                 # Python dependencies (5 packages)
|-- data/
|   |-- results_all_runs.csv          # Raw traces (200 records)
|   +-- results.json                  # Aggregated summary metrics
|-- figures/                         # 8 publication-quality PNG charts
|-- screenshots/                     # Terminal output screenshots
+-- docs/                            # PDF documentation
```

9. Execution Commands Quick Reference

Task	Command	Expected Output	Time
Setup environment	`make setup`	Creates .venv and installs all 5 dependencies	~30s
Run unit tests	`make test`	12 passed in 0.06s (all green)	<1s
Run 200 experiments	`make experiments`	results_all_runs.csv + results.json	~5s
Interactive CLI	`make cli`	Interactive scheduler with workload/policy selection	Interactive
Quick AFAS demo	`make demo`	AFAS decision log + performance metrics	~1s
Generate 8 charts	`make charts`	8 PNG figures in figures/ directory	~3s
Compile Review 2 PDF	`make report`	15-page academic PDF report	~2s
Run everything	**`make all`**	**Tests + experiments + charts + PDF**	**~10s**

Table 7: Complete command reference for running all project components.

10. Conclusion

This Review 3 report provides **comprehensive implementation evidence** for the Adaptive Fairness-Aware CPU Scheduling Simulator. The project is a **fully functional, rigorously tested** software system consisting of 6 Python source files (~1,800 lines of production code), 12 passing automated unit tests verified against hand-calculated analytical solutions, 200 benchmark simulation runs across 5 distinct workload families, and 8 publication-quality figures generated from the experimental data.

The technology stack (Python 3.14.3, Pytest 9.2.0, NumPy 2.5.4, Matplotlib 3.11.2, ReportLab 5.1.0) was selected for determinism, cross-platform portability, and scientific reproducibility. All configuration parameters, source code modules, and execution outputs have been documented with detailed explanations. The entire codebase, data, and documentation are publicly available on GitHub for faculty review and evaluation.

GitHub Repository: **https://github.com/AlokMaan/OS**

Review 2 PDF: OS_Review_2_Adaptive_Fairness_Aware_CPU_Scheduler.pdf (15 pages, in repo)
Review 3 PDF: OS_Review_3_AFAS_Implementation_Report.pdf (this document)